

Verification as a Library: Lessons from Sixteen Years of SBV

Experience Report

LEVENT ERKOK

SMT solvers make many verification problems accessible to programmers, but turning a program into a faithful solver query remains a substantial engineering task. SBV addresses this task as a Haskell library: programmers construct typed symbolic computations, state properties, and inspect counterexamples using ordinary Haskell abstractions. Developed since 2010, the library has grown from bit-vector verification to incremental solver interaction, symbolic algebraic datatypes, structured inductive proofs, and C generation. This report examines the architectural decisions and semantic boundaries exposed by that evolution. Three lessons recur: a symbolic embedding must make the boundary between host evaluation and logical computation explicit; incremental interaction makes declaration lifetime and dependency discovery part of the semantics of the API; and multiple interpretations of one symbolic representation require independent validation at their boundaries. We develop these lessons through executable examples, documented release changes, and the current regression infrastructure. The resulting account identifies both the reach of verification as a library and the obligations that remain with the programmer, the embedding, and the solver. It offers design guidance for Haskell libraries that connect a typed, functional interface to a stateful reasoning engine.

Additional Key Words and Phrases: Haskell, symbolic programming, SMT, embedded languages, verification, experience report

1 INTRODUCTION

An appealing way to introduce verification to a Haskell programmer is to start with a function and ask whether a property holds for every input. The notation can be almost familiar:

```
prove $ \x -> x + 1 .> (x :: SInteger)
-- Q.E.D.

prove $ \x -> x + 1 .> (x :: SInt8)
-- Counterexample: x = 127
```

The first claim concerns unbounded mathematical integers. The second concerns eight-bit signed arithmetic, where the successor of 127 is -128. The change of type changes the theorem. Neither a pleasant interface nor a powerful solver removes the need to choose the right semantics.

SBV [Erkok 2026] provides symbolic counterparts of Haskell types and operations for expressing such problems. A program over symbolic values constructs a representation that can be translated into SMT-LIB [Barrett et al. 2017], submitted to an external solver, and interpreted as a proof result or a model. The same infrastructure also supports constraint solving, optimization, interactive queries, and compilation of supported computations to C. Haskell supplies the host language, the type system, and much of the program-construction machinery.

The library began in 2010. Its release history records a first stable public Hackage release in January 2011, early C generation and changes to sharing, an incremental execution model in 2017, transformer support in 2019, and major extensions to proofs and datatypes in 2025–2026 [Erkok and SBV contributors 2026]. This history provides an opportunity to examine a question that is broader than any one feature: *which boundaries must an embedded verification library preserve as its capabilities grow?*

The answer developed here concerns three boundaries. First, Haskell evaluates the program that constructs a symbolic problem; symbolic operations describe the problem itself. Second, a typed expression exists within a particular symbolic run and, during a query, a particular evolving solver session. Third, concrete evaluation, SMT interpretation, and generated code must agree wherever

they claim to implement the same operation. Several apparently unrelated maintenance problems become understandable as failures to preserve one of these boundaries.

This report contributes an architectural account of the typed embedding and its shared representation; an analysis of documented changes to sharing, incremental execution, and datatype registration; and executable case studies connecting automated checking, inductive proof, and C generation. The contribution is experience with composing and maintaining these mechanisms. We do not claim a new SMT decision procedure, a new general technique for observable sharing, or a verified implementation of the library.

Our evidence is the 14.8 source snapshot, its release history, selected regression tests, and the small reproducibility companion supplied with this draft. The scope is deliberately explicit: examples demonstrate particular capabilities, while historical changes expose particular engineering pressures. Neither is a measurement of adoption or a comparative performance study. Section 7 states the limits of this evidence.

2 A SMALL PROGRAM WITH SEVERAL INTERPRETATIONS

Consider addition of unsigned bytes that saturates at 255 rather than wrapping. The implementation can test for overflow before adding. Its specification can use unbounded integers, making the intended mathematical result explicit:

```
satAdd :: SWord8 -> SWord8 -> SWord8
satAdd x y = ite (x .> 255 - y) 255 (x + y)

satAddSpec :: SWord8 -> SWord8 -> SBool
satAddSpec x y = sFromIntegral (satAdd x y) .==
    ite (total .> 255) 255 total
  where total = sFromIntegral x + sFromIntegral y :: SInteger
```

The conditional `ite` and equality `.==` operate on symbolic values. The specification converts each byte to an unbounded integer before adding. The expression `255 - y` does not underflow because every `SWord8` value is in the range 0 to 255. `prove satAddSpec` asks whether the two expressions agree for every pair of byte inputs. A successful result covers the finite input domain through the solver's reasoning; it is not a sample of concrete executions.

For literal inputs, operations can evaluate during construction. For example, `satAdd 250 10` yields the literal byte 255. For symbolic inputs, the operations build a graph. For C generation, the same definition supplies the body of a generated function:

```
generateAdder :: FilePath -> IO ()
generateAdder dir = compileToC (Just dir) "sat_add" $ do
  cgOverwriteFiles True
  cgGenerateDriver False
  x <- cgInput "x"
  y <- cgInput "y"
  cgReturn $ satAdd x y
```

The generator declares the input interface and the returned value. It does not invoke a solver at C run time. Moreover, proving `satAddSpec` does not establish correctness of the C compiler. The companion therefore also compiles the generated code and compares all 65,536 input pairs with an independently written C oracle using wider unsigned arithmetic. The theorem and the exhaustive execution check establish different facts.

Even a minimal example has several contracts. The input types specify the arithmetic domain. The property specifies the desired behavior. The embedding must encode both correctly. The solver

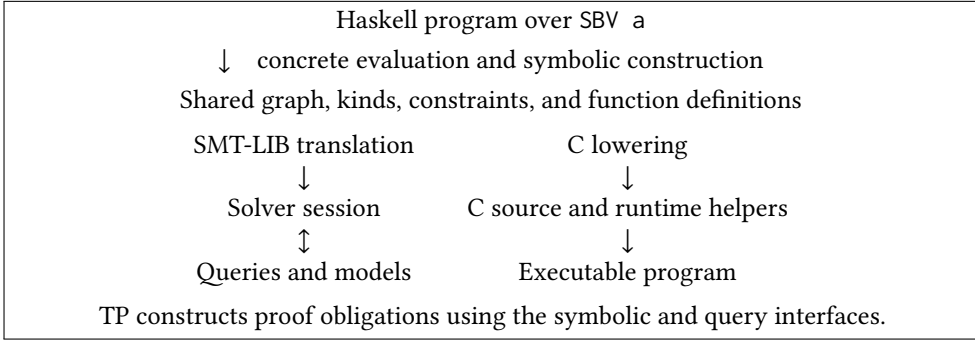


Fig. 1. The shared representation and its consumers. The arrows describe implementation flow, not a proved correctness relation between backends.

must answer the encoded question correctly. The code generator must implement the symbolic operations correctly under its documented C assumptions. Separating these contracts makes it possible to use one representation productively without attributing every guarantee to every consumer of that representation.

The familiar arithmetic syntax is selective. Haskell’s `Num` class allows overloading of addition and literals. Its `Eq` class returns `Bool`, so a symbolic equality needs a distinct operation returning `SBool`. Similarly, ordinary Haskell `if` inspects a concrete boolean; it cannot choose between branches on a symbolic condition. The resulting interface mixes ordinary operators with symbolic combinators. This visible distinction helps locate the boundary between constructing a problem and asking a logical question about it.

3 THE REPRESENTATION BEHIND THE INTERFACE

3.1 Typed values, concrete values, and graph nodes

The central public type is `SBV a`. For example, `SBool` is `SBV Bool`, `SInteger` is `SBV Integer`, and `SWord8` is `SBV Word8`. Internally, the representation has the following shape; this listing omits fields and constructors not needed for the discussion:

```

newtype SBV a = SBV { unSBV :: SVal }
data SVal = SVal Kind (Either CV (Cached SV))
data SV = SV Kind NodeId
data SBVExpr = SBVApp Op [SV]
newtype SymbolicT m a =
  SymbolicT { runSymbolicT :: ReaderT State m a }

```

The parameter of `SBV` is phantom in this representation but carries the public type information. `Kind` records the symbolic sort at run time; `CV` represents a concrete value. A non-concrete value contains a deferred computation that produces a symbolic node within a state. `SymVal` connects Haskell types to symbolic kinds and concrete representations.

This separation avoids making every internal graph operation a heterogeneous Haskell term indexed by its full type. It also creates implementation obligations: the phantom interface, run-time kinds, concrete payloads, and operation signatures must agree. The public API rules out many ill-typed combinations, but it is not a type-level proof of the implementation. Internal checks remain necessary, especially for casts and dynamically represented operations.

Concrete evaluation is embedded in the operation constructors. When arguments are known, an operation may produce a concrete result; otherwise it records an application of an operation to symbolic nodes. Algebraic simplifications can avoid graph construction as well. These simplifications must respect the operand kind. In particular, identities justified for integers cannot simply be reused for IEEE floating-point values, whose semantics includes NaNs, signed zero, and rounding.

3.2 Two forms of sharing

The implementation preserves sharing at two levels. Deferred symbolic computations are memoized within a symbolic state using stable names. Separately, `newExpr` consults a map of symbolic applications to reuse existing nodes. It normalizes certain argument orders and checks that a reused node has the requested kind. The latter check matters because an expression can participate in signedness-changing operations.

These mechanisms solve related but different problems. Reusing the result of a deferred computation avoids repeatedly constructing the same graph fragment. Reusing an existing application identifies equal constructed expressions even when their Haskell computations were separately allocated. Neither mechanism turns the graph into a canonical representation of mathematical equivalence: two equivalent formulas can still have different nodes.

Author material to add: a design decision. Describe one consequential choice in the embedding: the alternatives considered, the reason for the choice, and what subsequent experience changed. Sharing is a candidate, but the author should choose the episode. Do not infer personal motivations from the code.

The state contains mutable tables, counters, and accumulated assignments. `SymbolicT` presents this environment through a reader transformer, with mutation performed through references. `Symbolic` specializes the transformer to `IO`. The transformer interface allows clients to combine symbolic construction with their own effects; the ordinary interface keeps simple uses short. The 2019 release introduced this transformer support while retaining the usual entry points [Erkok and SBV contributors 2026].

3.3 Values belong to runs

Haskell type equality does not establish that two symbolic nodes belong to the same symbolic execution. Nodes therefore carry context information, and graph construction checks for common context mismatches. Returning a symbolic value from one solver run and combining it with a value from another can violate this invariant even when both have type `SInteger`.

This is a limitation of what the current public types express. In contrast, extracting a concrete Integer from a model and using it to construct a literal in a new run has a clear interpretation. The distinction suggests a general rule for embedded libraries: sharing the host type of a value does not establish the lifetime of the external resource to which that value refers. The implementation's context checks detect common violations, rather than providing a static guarantee for every possible misuse.

4 EVOLUTION EXPOSES THE BOUNDARIES

Table 1 selects changes relevant to the argument. The dates and descriptions are taken from the release history; the lessons below are an analysis of those changes and their current implementations.

Table 1. Selected milestones recorded in the release history [Erkok and SBV contributors 2026].

Date	Release	Change relevant to this report
2010	0.0.0–0.9.6	Initial infrastructure and design exploration
2011-03	0.9.12, 0.9.14	C generation; sharing reworked using stable names
2017-07	7.0	Interleaved symbolic execution and solver interaction
2019-01	8.0	Symbolic computations integrated with transformer stacks
2025-02	11.1	Reworked calculational and inductive proof interfaces
2025-10	13.0	User-defined symbolic algebraic datatypes
2026-09	14.8	New C backend; query and datatype registration fixes

4.1 Host evaluation is part of the embedding design

The March 2011 release history records replacing an earlier sharing mechanism with one based on stable names, inspired by `Data.Reify`. It also records problems involving constant applicative forms in the earlier approach. This is a concrete instance of the observable-sharing problem studied by Gill [2009]: a functional description can share a computation, while a naive reification expands it into a much larger tree.

For SBV, the practical requirement is to recover reusable structure without making the ordinary arithmetic interface monadic at every operation. The current design defers node creation until a symbolic state is available and memoizes within that state. The historical record supports the need to revise this mechanism; it does not by itself quantify the performance improvement.

Recursion reveals the same host/object distinction from a different direction. A Haskell function that recursively calls itself under `ite` may continue constructing symbolic branches indefinitely. A finite input type does not guarantee a finite construction: unless the stopping condition becomes concrete during host evaluation, the construction can keep unfolding. The Fibonacci code-generation example in the repository illustrates using an explicit construction bound to obtain a finite graph.

Named symbolic functions provide another route. `smtFunction` represents a function definition and its calls at the SMT level, allowing recursive definitions to remain recursive rather than being fully unfolded by Haskell. That changes where the recursion is represented; it also introduces obligations about definitions and termination. The library can help discharge those obligations, but moving a recursive call into the solver is not itself a proof that the function is well founded.

4.2 Incremental solving changes the lifetime of declarations

The 2017 release documents a change from running a symbolic program before invoking the solver to interleaving execution with solver interaction. Its stated motivation was to let clients send and receive solver commands instead of being limited to predefined recipes. The earlier tactic interface was removed as part of that change [Erkok and SBV contributors 2026].

The query interface makes it possible to inspect a model and decide what to ask next. The following example enumerates the integer roots of a polynomial:

```

roots :: IO [Integer]
roots = runSMT $ do
  x <- sInteger "x"
  constrain $ x * x .== 9
  query $ let loop = do
    result <- checkSat
    case result of
      Sat -> do v <- getValue x
                constrain $ x ./= literal v
                (v :) <$> loop
      Unsat -> pure []
    _ -> io $ fail "Root enumeration was inconclusive"
  in loop

```

After a satisfiable result, the program obtains a concrete root and adds a constraint excluding it. The next check runs in the same session. Enumeration ends when the remaining constraints are unsatisfiable. An inconclusive solver response is handled separately; it is not treated as exhaustion. Root order is not part of the contract.

Such a small loop has consequences for the implementation. Literals, expressions, types, and functions can be encountered after the initial problem has been sent. The library must record newly created objects and synchronize them with the solver before a command refers to them. Its symbolic state therefore maintains an incremental component as well as the full accumulated program. Query operations use that component to emit new declarations, definitions, assignments, and constraints.

Release 14.8 fixes first use of `smtFunction` definitions during queries without prior explicit registration. It also fixes first use of rational kinds in query mode, including occurrences nested inside other types. These changes demonstrate that incremental support is not complete merely because `checkSat` and `getValue` are available. Every operation that can introduce a dependency must participate in synchronization.

The resulting lesson is about API compositionality. If an expression can be constructed during setup and during a query, its dependencies should normally follow it in either context. Requiring an unrelated setup-time use to make a later expression work exposes an implementation accident. Some solver options genuinely must be chosen before a session starts; those restrictions should be distinguished from missing dependency registration.

Author material to add: a historical surprise. Add a specific user request, solver behavior, or failed design that changed the direction of SBV. Include an approximate date and, where available, a release or issue reference. Explain the resulting lesson rather than only recounting the event.

4.3 Datatypes make dependencies structural

SBV uses Template Haskell to make user-defined datatypes symbolic. A splice such as `mkSymbolic [' 'Box]` generates the symbolic infrastructure for a Haskell datatype. `sCase` provides symbolic pattern matching, including nested patterns and guards. The mechanism works with the embedding's symbolic representation; it does not change the meaning of arbitrary Haskell case expressions throughout a program.

A datatype dependency need not occur immediately in a constructor field. It can appear inside a tuple, a list, a set, an array, or a type argument. Mutually recursive definitions add cycles. Consequently, registering only the visible root constructor is insufficient. The library must discover the structural dependencies needed to declare and use the symbolic kind.

The 14.8 release records fixes for precisely these cases. The associated registration tests vary both the position of a nested datatype and the phase in which a symbolic value is first created: setup, a named query variable, or an unnamed query variable. This forms a useful regression matrix. A test of one nested datatype in a conventional setup block cannot establish that first use inside a query will work.

An abbreviated example in the companion creates two `Box Word8` values for the first time inside a query and requires them to differ. The result should be satisfiable. The full repository tests extend that idea to container nesting, synonyms, and recursive dependencies. The general lesson is to test a feature at the places where it enters the system, in addition to testing the feature's operations once it is already registered.

5 FROM AUTOMATED CHECKS TO STRUCTURED PROOFS

Push-button checking is effective when the solver can discover the necessary argument. Recursive functions often need structure supplied by the user: an induction principle, a stronger invariant, a case split, or intermediate equalities. SBV's TP layer lets that structure be written as a Haskell program that generates and checks proof obligations.

An accumulator-based list reversal illustrates the interaction. Its useful invariant quantifies over both the input list and the accumulator:

```
revAcc :: SList Integer -> SList Integer -> SList Integer
revAcc = smtFunction "paper.revAcc" $ \acc xs ->
  [sCase| xs of
    []      -> acc
    x : xt -> revAcc (x .: acc) xt
  |]

revAccCorrect :: TP (Proof (Forall "xs" [Integer]
                             -> Forall "acc" [Integer] -> SBool))
revAccCorrect = induct "revAccCorrect"
  (\(Forall xs) (Forall acc) ->
    revAcc acc xs == SL.reverse xs SL.++ acc) $
  \ih (x, xs) acc -> [] |- revAcc acc (x .: xs)
    =: revAcc (x .: acc) xs
    ?? ih
    =: SL.reverse xs SL.++ (x .: acc)
    =: (SL.reverse xs SL.++ [x]) SL.++ acc
    =: SL.reverse (x .: xs) SL.++ acc
    =: qed
```

The function is specialized to symbolic lists of integers here to keep the listing small. `Forall` names quantified parameters at the type level. `induct` supplies the list induction structure, including a base case and an induction hypothesis. The user gives the calculation for the step case. The operators `|-`, `=:`, and `??` introduce assumptions, successive expressions, and proof hints respectively.

The crucial human contribution is the generalized statement involving `acc`. A claim only about reversing into an empty accumulator is too weak to apply directly to the recursive call. Once the stronger statement is chosen, the calculation exposes the useful intermediate expressions and gives the solver smaller obligations. The notation expresses a proof strategy; Haskell's type checker alone does not certify the displayed equalities.

In ordinary checked execution, TP’s proof-step machinery negates the requested proposition under the current assumptions and asks for satisfiability. Unsatisfiability discharges the step. A satisfiable or inconclusive result does not. The layer records proof dependencies and supports reuse of lemmas. Recursive definitions additionally require attention to their termination measures. The library offers measure-based checks and records uses of functions whose termination checks were explicitly bypassed.

The constant-folding development is a larger example already shipped with the library. Its expression datatype includes variables, constants, arithmetic, and local bindings. The transformation recursively folds children and then simplifies their parent. Replacing a binding by a constant requires substitution that respects shadowing. Its main claim is:

$$\forall e, \rho. \quad \text{interpInEnv}(\rho, \text{cfold}(e)) = \text{interpInEnv}(\rho, e).$$

The proof uses a size measure and auxiliary lemmas about substitution, environments, and arithmetic. This example makes an important limitation visible: the convenience of an embedded prover does not eliminate the work of discovering the supporting theory. The paper companion invokes the existing development; it does not claim a new constant-folding algorithm or a new proof of that algorithm.

These are claims about the represented semantics. `SList a` denotes a finite sequence of values of type `a`; it is not the domain of arbitrary lazy Haskell lists with partial and infinite values. A reversal theorem over symbolic lists therefore does not establish the corresponding statement for all Haskell list computations. Likewise, an interpreter represented with symbolic datatypes has the semantics of its symbolic definitions. Relating it to a separately implemented interpreter is a further verification task.

6 CODE GENERATION TESTS THE COMMON REPRESENTATION

C generation has been present since 2011, but the 14.8 backend substantially extends its scope. It supports, among other values, arbitrary-width bit-vectors, exact integers, rational-valued reals, floating-point values, structured values, and recursive function definitions. Exact unbounded integers and rational-valued reals use GMP; some floating-point operations require LibBF. The legacy backend remains separately available.

Sharing the expression graph avoids repeating the front-end representation, but the target has different operational requirements. A solver reasons about mathematical terms. C needs a choice of evaluation order, storage, ownership, and failure behavior. The backend therefore lowers an expression to a result expression together with declarations, setup statements, and required runtime facilities. Setup statements must execute when their values are needed. Moving them across conditional branches indiscriminately can change behavior.

Numeric operations expose another boundary. The release notes record fixes to pseudo-Boolean comparisons whose totals exceed the range of a naive native accumulator, and to flooring of native-mapped reals when converting to integers. Both operations can be easy to express symbolically while being subtle to implement in C. The regression suite uses independent mathematical expectations and executes generated code, including checks at different optimization levels and with sanitizers in selected tests.

The generator has explicit limits. Quantifiers and other solver-only constructs are rejected. Exact real support covers rational values rather than arbitrary algebraic reals. Direct array equality is supported only for enumerable finite key domains within a configured budget. Floating-point generation has documented rounding and compiler-option requirements. These boundaries are part of the meaning of using the backend, not implementation details that can be hidden behind a common API.

Table 2. Companion validation tasks. These are semantic checks, not timing benchmarks; the execution record accompanies the draft.

Task	Required outcome
Integer successor	Theorem holds over unbounded integers
Signed-byte successor	Counterexample is the value 127
Floating-point reflexivity	A NaN is returned as a counterexample
Saturating addition	Implementation satisfies the integer specification
Incremental roots	Exactly -3 and 3 , followed by unsatisfiability
Late datatype declaration	Distinct query-created byte boxes are satisfiable
Accumulator reversal	TP completes the generalized inductive proof
Constant folding	The existing CVC5 proof development completes
Generated C addition	All 65,536 input pairs agree with a wider-arithmetic oracle

The saturating-adder case in Section 2 is intentionally small enough to validate exhaustively. It shows one way to test the connection between the specification, symbolic implementation, and generated code. It supplies no blanket guarantee for the other operations or runtime representations.

7 EVIDENCE AND REPRODUCIBILITY

The source snapshot used here is commit `f4f7a7a08445e63c2fb74c6f2084dfea5ad8b7d2`, identifying release 14.8. The companion’s `README.md` records build commands and the local tool versions; `results/validation.txt` records the executed checks. The paper’s main executable listings are extracted from the checked Haskell source rather than maintained as separate copies.

7.1 Small, explicit validation tasks

Table 2 defines the companion’s tasks and the evidence each supplies. All nine tasks completed successfully on 26 September 2026 using GHC 9.14.1, Z3 5.1.0, and CVC5 1.4.0 on an ARM64 macOS host. The generated C was compiled with Apple Clang 21.0.0, optimization enabled, strict warnings, and undefined-behavior sanitization. The checks cover distinct paths through the library. Their small scale is deliberate: they make the claims in the text reproducible without turning an experience report into a broad solver benchmark.

The companion checks model values rather than assuming a particular printed model order. It treats timeouts as failures to complete a task, not as evidence that a proposition is false. It also checks the generated adder against an oracle whose arithmetic domain is wider than the inputs. These choices keep the evidence close to the semantic claim being made.

7.2 Regression infrastructure as experience evidence

The repository’s testing infrastructure combines HUnit tests, QuickCheck properties, golden files, compile-time diagnostics, executable documentation, and generated-C integration tests. Each addresses a different failure mode. The shared `qc1` and `qc2` helpers, for example, can exercise an operation both on concrete symbolic literals and through a solver. This is a direct response to the need for concrete evaluation and SMT translation to agree.

Golden files make changes to models, proof transcripts, and generated text reviewable. They are valuable records of behavior, but their evidential force depends on what was recorded and reviewed. A stable transcript is not an independent proof that the translation was correct. Similarly,

compile-failure tests for sCase and pCase exercise the diagnostics and rejection behavior of the metaprogramming interface; they address concerns that a successful solver example cannot test.

The datatype-registration matrix and executed C boundary tests are especially useful examples of tests derived from architectural obligations. They vary the phase, nesting, representation, or compilation mode that previously exposed a problem. That is more informative than demonstrating a newly added operation only in an already well-prepared context.

7.3 Limits of the account

Author material to add: an application outside the example suite. Describe a real use of SBV: the problem, why this approach was chosen, what worked, what did not, and evidence that can be cited or reproduced. Identify any limitations on naming the application. This material will extend the present account beyond examples and maintenance records.

The examples and regressions were selected to explain the three boundaries identified in Section 1. They are not a random sample of all uses of SBV, and they do not estimate failure rates. The release notes provide contemporaneous records of changes, but not a controlled study of their causes or effects. We make no measured claim that the API reduces development effort, that every supported solver behaves identically, or that the current implementation is free of semantic bugs.

The source contains many larger examples, but the existence of an example is not evidence that arbitrary problems in the same domain will be tractable. Quantifiers, nonlinear arithmetic, and recursive definitions can make solver behavior difficult to predict. Different solver releases can require different proof guidance. The successful small checks establish only their stated outcomes in the recorded environment.

8 RELATED WORK

Properties as Haskell programs. QuickCheck [Claessen and Hughes 2000] established an influential style in which programmers express properties in Haskell and use generated inputs to seek counterexamples. SBV shares the appeal of a library-level interface but constructs logical formulas for supported symbolic operations. Testing and SMT reasoning offer different forms of evidence and can be used together, as they are within SBV’s own regression suite. The syntax of a property alone does not determine its evidential strength.

Reification and sharing. Gill [2009] develops type-safe observable sharing for embedded languages. SBV’s release history explicitly connects its stable-name implementation to Data.Reify. Our account concerns how deferred construction, expression reuse, and per-run state coexist in this particular library. It does not present observable sharing as a new technique.

High-level symbolic computation. Smten [Uhler and Dave 2013] translates high-level symbolic computations into SMT queries, addressing the burden of manually constructing effective formulas. Rosette’s symbolic virtual machine [Torlak and Bodik 2014] supports solver-aided host languages and uses symbolic evaluation to make rich program descriptions available to solvers. These systems share the goal of raising the level at which a user describes a reasoning problem. The relevant distinction here is the boundary exposed by a Haskell library whose symbolic values and operations are explicit in the host program. This report does not attempt a feature or performance ranking of these systems.

Grisette [Lu and Bodik 2023] is a particularly close comparison: it provides symbolic compilation as a typed functional library in Haskell, with a purely functional representation and an ordered-guards approach to state merging. Its work demonstrates that a library embedding is itself an

established research direction. SBV’s contribution here is a longitudinal account of a different engineering composition: deferred graph construction with per-run state, incremental SMT sessions, structured proof orchestration, and a C consumer of the symbolic representation. A direct performance comparison would require carefully matched workloads and is outside this report.

SMT-assisted deductive proofs. Refinement reflection [Vazou et al. 2018] connects user-defined functions to SMT reasoning within Liquid Haskell’s refinement-type verification framework. TP instead presents explicit proof programs whose obligations are discharged through SBV’s symbolic and query interfaces. Both approaches let users supply structure that automated reasoning alone may not discover. Their relationship calls for a comparison of specification styles, trusted components, and proof guidance, rather than an assumption that all SMT-assisted proofs provide the same interface or guarantees.

Solvers and their interface. SMT-LIB [Barrett et al. 2017] provides the language through which SBV communicates with its backends. Z3 [de Moura and Bjørner 2008] is the default solver, with other solvers selected through configurations and capability descriptions. SBV relies on these engines for the logical reasoning. Its responsibilities include choosing faithful encodings, maintaining the protocol, and interpreting responses without converting an inconclusive outcome into a proof.

9 DISCUSSION AND LESSONS

Verification as a library provides a useful division of labor. Haskell handles program construction, abstraction, and much of the static discipline. SBV records symbolic meaning and manages the connection to external reasoning. Solvers discharge the resulting obligations. The experience examined here suggests three lessons for similar libraries.

First, expose semantic differences at the point where users make choices. Separate symbolic equality and branching, distinguish a symbolic sequence from a Haskell container of symbolic elements, and state the domain of a theorem. Familiar notation is useful when its interpretation is clear.

Second, treat incremental execution as a property of the whole interface. Registration, recursive definitions, constants, and nested types must work when first encountered during a live session. Tests should vary the phase of first use as systematically as they vary operand types.

Third, use a shared representation to organize independent checks of its interpretations. Concrete evaluation, SMT translation, and generated C can cross-check selected operations, but common implementation structure can also propagate a mistake. Independent mathematical specifications and execution oracles are therefore valuable at semantic boundaries.

The trusted base remains substantial: the Haskell implementation and its runtime, the symbolic encoding, proof orchestration and any supplied axioms, solver behavior, and, where relevant, the C compiler and generated runtime. TP proof objects record propositions and dependencies; the ordinary workflow does not establish every result by replaying a certificate in an independent small kernel. This account should be read with that trust model in mind.

10 CONCLUSION

Sixteen years of SBV development show how a Haskell library can grow from bit-vector properties to interactive reasoning and structured proofs while retaining a compact programming interface. The documented changes also show where the cost of that growth appears: host evaluation and sharing, the lifetime of solver declarations, and agreement between interpretations. Making those boundaries explicit yields both a clearer user model and more targeted tests. Those lessons are applicable to embedded libraries that must preserve a typed functional interface while coordinating stateful external tools.

REFERENCES

- Clark Barrett, Pascal Fontaine, and Cesare Tinelli. 2017. *The SMT-LIB Standard: Version 2.6*. Technical Report. Department of Computer Science, The University of Iowa. <https://smt-lib.org/language.shtml> The language site also provides subsequent revisions.
- Koen Claessen and John Hughes. 2000. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the ACM SIGPLAN International Conference on Functional Programming*. ACM, New York, NY, USA, 268–279. <https://research.chalmers.se/en/publication/237427>
- Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems*. Springer, Berlin, Heidelberg, 337–340. https://doi.org/10.1007/978-3-540-78800-3_24
- Levent Erkok. 2026. SBV: SMT Based Verification in Haskell. Software, version 14.8. <https://github.com/LeventErkok/sbv/tree/f4f7a7a08445e63c2fb74c6f2084dfea5ad8b7d2> Source snapshot examined in this report.
- Levent Erkok and SBV contributors. 2026. SBV Release History. <https://github.com/LeventErkok/sbv/blob/f4f7a7a08445e63c2fb74c6f2084dfea5ad8b7d2/CHANGES.md> Versions 0.0.0–14.8.
- Andy Gill. 2009. Type-Safe Observable Sharing in Haskell. In *Proceedings of the Second ACM SIGPLAN Symposium on Haskell*. ACM, New York, NY, USA, 117–128. <https://doi.org/10.1145/1596638.1596653>
- Sirui Lu and Rastislav Bodik. 2023. Griset: Symbolic Compilation as a Functional Programming Library. *Proceedings of the ACM on Programming Languages* 7, POPL, Article 16 (2023), 33 pages. <https://doi.org/10.1145/3571209>
- Emina Torlak and Rastislav Bodik. 2014. A Lightweight Symbolic Virtual Machine for Solver-Aided Host Languages. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, New York, NY, USA, 530–541. <https://doi.org/10.1145/2594291.2594340>
- Richard Uhler and Nirav Dave. 2013. Smten: Automatic Translation of High-Level Symbolic Computations into SMT Queries. In *Computer Aided Verification*. Springer, Berlin, Heidelberg, 678–683. https://doi.org/10.1007/978-3-642-39799-8_45
- Niki Vazou, Anish Tondwalkar, Vikraman Choudhury, Ryan G. Scott, Ryan R. Newton, Philip Wadler, and Ranjit Jhala. 2018. Refinement Reflection: Complete Verification with SMT. *Proceedings of the ACM on Programming Languages* 2, POPL, Article 53 (2018), 31 pages. <https://doi.org/10.1145/3158141>